

## Contents

KPFC Admin SSO Integration Guide for Fleet

Responsibility boundary

Endpoints and environments

What KPFC Admin will provide

Fleet implementation setup

Authorization Code with PKCE

Profile contract

Logout, revocation, and introspection

Lifecycle webhook

Laravel client outline

Certification checklist

Troubleshooting

Rotation and contract changes

# KPFC Admin SSO Integration Guide for Fleet

---

**Audience:** The developer building and maintaining `fleet.kpfcbuilders.com`. This document describes what to implement on the Fleet side and the information KPFC Admin will provide.

KPFC Admin is the identity provider. Fleet is an OAuth 2.0 confidential client and keeps a local shadow user keyed only by the immutable Admin `sub`.

The Fleet application must never collect an Admin password, copy Admin password hashes, share Admin session cookies, or query the Admin database directly. User SSO tokens must also remain separate from any machine-to-machine Fleet business API credentials.

## Responsibility boundary

| KPFC Admin team provides                       | Fleet developer implements                                   |
|------------------------------------------------|--------------------------------------------------------------|
| OAuth issuer and endpoints                     | Sign-in button and redirect                                  |
| Environment-specific client ID and secret      | Secure server-side secret and token storage                  |
| Exact callback registration                    | Callback route with <code>state</code> and PKCE verification |
| <code>access_fleet_system</code> authorization | Local shadow user keyed by immutable <code>sub</code>        |
| Profile, revocation, and introspection APIs    | Local session, refresh, logout, and error handling           |
| Signed user lifecycle events                   | Verified, replay-safe webhook receiver                       |

## Endpoints and environments

| Purpose                | Staging                                             | Production                                  |
|------------------------|-----------------------------------------------------|---------------------------------------------|
| Issuer                 | <code>https://admin-staging.kpfcbuilders.com</code> | <code>https://admin.kpfcbuilders.com</code> |
| Authorization          | <code>/oauth/authorize</code>                       | <code>/oauth/authorize</code>               |
| Token exchange/refresh | <code>/oauth/token</code>                           | <code>/oauth/token</code>                   |
| User profile           | <code>/api/sso/user</code>                          | <code>/api/sso/user</code>                  |
| Token revocation       | <code>/api/oauth/revoke</code>                      | <code>/api/oauth/revoke</code>              |
| Token introspection    | <code>/api/oauth/introspect</code>                  | <code>/api/oauth/introspect</code>          |

Use a different OAuth client, secret, webhook secret, and callback URI in each environment. Never put production credentials in staging.

## What KPFC Admin will provide

Before Fleet integration begins, the KPFC Admin team will securely provide:

- The staging issuer URL: `https://admin-staging.kpfcbuilders.com`.
- A staging OAuth client ID and client secret.
- Confirmation that Fleet's exact staging callback URI has been registered.
- The required scopes: `fleet:login` `fleet:profile`.

- A staging webhook signing secret.
- Confirmation that the test user has `access_fleet_system`.

Production will use a different client, client secret, callback registration, and webhook secret. Credentials will be delivered through the password manager, not email, chat, source control, or this document.

The Fleet developer does **not** need server access to KPFC Admin and should not run Admin migrations, generate Passport keys, create OAuth clients, or edit the Admin environment. Those are KPFC Admin team responsibilities.

If any item above is missing, request it from the KPFC Admin operations owner before starting end-to-end testing.

## Fleet implementation setup

---

```
KPFC_SSO_ISSUER=https://admin-staging.kpfcbuilders.com
KPFC_SSO_CLIENT_ID=<delivered-separately>
KPFC_SSO_CLIENT_SECRET=<delivered-separately>
KPFC_SSO_REDIRECT_URI=https://fleet-staging.kpfcbuilders.com/auth/kpfc/callback
KPFC_SSO_SCOPES="fleet:login fleet:profile"
KPFC_SSO_WEBHOOK_SECRET=<delivered-separately>
```

Store secrets encrypted or in the deployment secret manager. Do not expose the client secret to browser JavaScript, logs, exception pages, source control, or CI artifacts.

## Authorization Code with PKCE

---

1. Fleet generates a cryptographically random `state` and PKCE `code_verifier`, stores both in the server-side login session, and derives:

```
code_challenge = BASE64URL(SHA256(code_verifier))
```

2. Redirect the browser:

```
GET {issuer}/oauth/authorize
  ?client_id={client_id}
  &redirect_uri={exact_callback}
  &response_type=code
```

```
&scope=fleet%3Alogin%20fleet%3Aprofile
&state={random_state}
&code_challenge={challenge}
&code_challenge_method=S256
```

3. Admin authenticates the user and verifies `access_fleet_system`. The user approves the requested scopes. Admin redirects to the exact callback with `code` and `state`.
4. Fleet compares `state` with the session using a timing-safe comparison, consumes it once, then exchanges the code from its backend:

```
curl -X POST "$KPFC_SSO_ISSUER/oauth/token" \
  -H 'Accept: application/json' \
  -d grant_type=authorization_code \
  -d client_id="$KPFC_SSO_CLIENT_ID" \
  -d client_secret="$KPFC_SSO_CLIENT_SECRET" \
  -d redirect_uri="$KPFC_SSO_REDIRECT_URI" \
  -d code="$CODE" \
  -d code_verifier="$CODE_VERIFIER"
```

5. Store access and refresh tokens encrypted server-side. Never put them in local storage or a browser-readable cookie. Fetch the profile, upsert the Fleet shadow user by `sub`, create Fleet's own secure session, and redirect away from the callback URL.

Access tokens last 15 minutes. Refresh tokens last 30 days and rotate on use:

```
curl -X POST "$KPFC_SSO_ISSUER/oauth/token" \
  -H 'Accept: application/json' \
  -d grant_type=refresh_token \
  -d client_id="$KPFC_SSO_CLIENT_ID" \
  -d client_secret="$KPFC_SSO_CLIENT_SECRET" \
  -d refresh_token="$REFRESH_TOKEN" \
  -d scope='fleet:login fleet:profile'
```

Persist the newly returned refresh token atomically before discarding the old one. Concurrent refreshes using the same token are invalid.

## Profile contract

```
curl "$KPFC_SSO_ISSUER/api/sso/user" \
-H "Authorization: Bearer $ACCESS_TOKEN" \
-H 'Accept: application/json'
```

Success ( 200 ):

```
{
  "sub": "bca7909f-cdf0-4f59-a69d-6cb61f7d34a6",
  "name": "Leah Example",
  "email": "leah@example.com",
  "phone": "+254700000000",
  "role": "admin",
  "fleet_access": true,
  "updated_at": "2026-09-21T09:00:00+03:00"
}
```

`sub` is the only account-linking key. It never changes. Email, phone, name, and role are mutable attributes and must never be used to merge users.

| Status                                 | Meaning                                                            |
|----------------------------------------|--------------------------------------------------------------------|
| 401                                    | Missing, invalid, expired, or revoked bearer token                 |
| 403 / <code>fleet_access_denied</code> | User or OAuth client is not authorized for Fleet                   |
| 403 missing scope                      | Token lacks <code>fleet:login</code> or <code>fleet:profile</code> |
| 429                                    | Rate limit reached; retry with backoff                             |

## Logout, revocation, and introspection

On Fleet logout, destroy the Fleet session and revoke the access token. Use HTTP Basic authentication for both endpoints:

```
curl -X POST "$KPFC_SSO_ISSUER/api/oauth/revoke" \
-u "$KPFC_SSO_CLIENT_ID:$KPFC_SSO_CLIENT_SECRET" \
-d token="$ACCESS_TOKEN"
```

Revocation is idempotent and returns `200` even when the token is already unknown or revoked.

Fleet may validate a session before sensitive operations:

```
curl -X POST "$KPFC_SSO_ISSUER/api/oauth/introspect" \  
-u "$KPFC_SSO_CLIENT_ID:$KPFC_SSO_CLIENT_SECRET" \  
-d token="$ACCESS_TOKEN"
```

An active response contains `active`, `client_id`, `sub`, `scope`, `iat`, and `exp`. Every invalid, expired, revoked, deleted, unauthorized, or foreign-client token produces only `{"active":false}`. A bad client credential returns `401` with `error=invalid_client`.

## Lifecycle webhook

Admin sends `user.updated`, `user.disabled`, `user.restored`, and `user.access_revoked` events to `FLEET_SSO_WEBHOOK_URL`. The body is:

```
{  
  "id": "108d5874-751a-4baa-aef4-259acc61ba7d",  
  "event": "user.access_revoked",  
  "subject": "bca7909f-cdf0-4f59-a69d-6cb61f7d34a6",  
  "occurred_at": "2026-09-21T09:10:00+03:00",  
  "user": {  
    "sub": "bca7909f-cdf0-4f59-a69d-6cb61f7d34a6",  
    "name": "Leah Example",  
    "email": "leah@example.com",  
    "phone": "+254700000000",  
    "fleet_access": false  
  }  
}
```

Headers:

```
X-KPFC-Event-Id: <same UUID as body.id>  
X-KPFC-Timestamp: <Unix seconds>  
X-KPFC-Signature: v1=<lowercase hex HMAC-SHA256>
```

Fleet must:

1. Reject timestamps more than five minutes from its clock.
2. Compute `hash_hmac('sha256', timestamp + "." + exact_raw_body, secret)`.
3. Compare the expected `v1=` value with `hash_equals`.
4. Insert `X-KPFC-Event-Id` into a unique event-receipt table before applying the event. Return `2xx` for an already processed ID.
5. Terminate every Fleet session and discard stored tokens immediately for `user.disabled` and `user.access_revoked`.

6. Upsert mutable profile data by `subject` for update/restore events.

Admin retries non-2xx responses after 60, 300, and 900 seconds. Delivery state is retained in `fleet_sso_events`; operations must investigate `failed` rows.

## Laravel client outline

---

Use Laravel's HTTP client from Fleet's backend. Keep state, verifier, tokens, and secrets server-side:

```
$token = Http::asForm()->post(config('services.kpfc_sso.issuer').'/oauth/token', [
    'grant_type' => 'authorization_code',
    'client_id' => config('services.kpfc_sso.client_id'),
    'client_secret' => config('services.kpfc_sso.client_secret'),
    'redirect_uri' => route('auth.kpfc.callback'),
    'code' => $request->string('code')->toString(),
    'code_verifier' => $request->session()->pull('kpfc_code_verifier'),
])->throw()->json();

$profile = Http::withToken($token['access_token'])
    ->get(config('services.kpfc_sso.issuer').'/api/sso/user')
    ->throw()
    ->json();
```

Validate `state` before this exchange. Wrap the local-user upsert and encrypted token persistence in one transaction.

## Certification checklist

---

Before enabling production, verify against staging:

- First login prompts for consent and creates exactly one shadow user.
- Repeat login links to the same local user by `sub`, even after email change.
- A callback with wrong `state`, verifier, callback URI, or reused code fails.
- Missing either required scope prevents profile access.
- Refresh rotates the refresh token; reuse of the old value fails.
- Fleet logout revokes the token and destroys the Fleet session.
- Introspection changes from active to inactive after revocation.
- Deleting the Admin user or removing `access_fleet_system` invalidates tokens, sends a signed event, and terminates Fleet sessions.
- A duplicated webhook event ID has no second effect.

- An old timestamp or invalid signature is rejected.
- No password, private key, client secret, live token, or sensitive production user record appears in source control, logs, screenshots, or test fixtures.

## Troubleshooting

| Symptom                            | Check                                                                          |
|------------------------------------|--------------------------------------------------------------------------------|
| <code>redirect_uri</code> mismatch | Compare scheme, host, path, slash, and case with the exact registered callback |
| <code>invalid_client</code>        | Correct environment client ID/secret; rotate if exposed                        |
| State or PKCE failure              | One-time server session values, S256 challenge, no verifier truncation         |
| Missing scope                      | Request exactly <code>fleet:login fleet:profile</code> ; obtain a new grant    |
| <code>fleet_access_denied</code>   | Grant <code>access_fleet_system</code> to the user's current Admin role        |
| Expired/revoked token              | Refresh once; if refresh fails, restart login                                  |
| Webhook signature failure          | Verify the raw body before JSON parsing and use the same environment secret    |
| Webhook timestamp failure          | Synchronize both hosts with NTP; retain the five-minute tolerance              |
| Repeated webhook retries           | Return <code>2xx</code> after committing the idempotent event receipt          |

## Rotation and contract changes

Rotate a client by creating a replacement, deploying its credentials to Fleet, testing login, then revoking the old client and its tokens. Rotate webhook secrets with a short dual-secret verification window on Fleet, then remove the old secret.

Endpoint or payload changes must be backward compatible or introduced under a new API version. Update this runbook and its examples in the same change. Escalate failed authentication or webhook delivery through the KPFC Admin operations owner; include request/event IDs, timestamps, and HTTP statuses, never tokens or secrets.